

TERESA — Braccio Unitree Z1

Diario di sviluppo, TODO list e guida di avvio

Branch: `FSM_main` | Repo: `TERESA_ws/src/z1_vision`

Ultimo aggiornamento: **15 Marzo 2026**

1 Panoramica del sistema

Il sistema controlla il braccio **Unitree Z1** montato sul robot TERESA per avvicinarsi in modo autonomo al torso di un paziente. Lo stack completo è composto da:

- **Percezione:** YOLO11 pose estimation + Kalman 3D + lock tracker
- **Pianificazione:** Workspace checker (Pinocchio + L-BFGS-B)
- **Controllo posizione:** IK Jacobiana pseudo-inversa smorzata + JointTrajectoryController
- **Controllo forza:** Impedance controller in spazio cartesiano (500 Hz, torque control)
- **Sicurezza:** keyboard safety node, stati HOMING/EMERGENCY, workspace clipping
- **Orchestrazione:** FSM a 11 stati, launch files separati per controllo e percezione

2 Architettura FSM (stato attuale)

Stati della FSM (`z1_FSM.py`)

Stato	Descrizione
HOMING	Stato iniziale al boot. Invia home pose via IK/JTC, poi va in WAITING
WAITING	Aspetta target LOCKED fresco (<0.5s) dal tracker YOLO
CHECKING_WORKSPACE	Verifica raggiungibilità target (thread non bloccante, Pinocchio)
APPROACHING	Invia goal IK verso il target (latchato al momento del check)
WAIT_IK_DONE	Attende <code>/ik_done=True</code> dal nodo IK
SWITCHING_TO_TORQUE	Switch JTC → torque_controller via <code>safe_controller_switch</code>
IMPEDANCE_RUNNING	Impedance controller attivo: approach → hold → retract
SWITCHING_TO_JTC	Switch torque_controller → JTC
EMERGENCY_SWITCHING	Forza switch a JTC se torque attivo, poi va in HOMING
EMERGENCY	Stato bloccante, aspetta solo comando <code>reset</code> da tastiera
FAULT	Errore fatale (switch fallito), nodo bloccato

Topic principali

Topic	Tipo	Descrizione
/torso_target_ee_locked	PoseStamped	Target EE quando tracker LOCKED
/torso_target_ee	PoseStamped	Target EE interpolato continuo
/ik_goal_pose	PoseStamped	Goal IK da FSM
/ik_enable	Bool	Abilita nodo IK
/ik_done	Bool	Conferma fine movimento JTC
/impedance_enable	Bool	Abilita impedance controller
/impedance_done	Bool	Fine ciclo impedance
/target_out_of_workspace	Bool	Target clippato dal workspace checker
/z1_fsm/state	String	Stato corrente FSM
/z1_keyboard_cmd	String	Comandi da tastiera (home/emergency/reset)
/torso_surface_frame	PoseStamped	Piano superficie torso (PCA)
/surface_signed_distance	Float32	Distanza TCP-superficie

3 DONE — Funzionalità implementate

Funzionalità completate al 2026-03-15

Pipeline base (commit precedenti)

- ✓ Nodo `z1_yolo_torso_tracker`: YOLO11 pose estimation, Kalman 3D $[x,y,z,vx,vy,vz]$ con dt adattivo, FSM IDLE/ESTIMATING/LOCKED, lock su varianza 20 frame + 5 check consecutivi
- ✓ Nodo `z1_ik_to_jtc`: IK Jacobiana pseudo-inversa smorzata (frame LOCAL), smoothstep quintico ($10t^3 - 15t^4 + 6t^5$), joint unwrap anti-flip, interfaccia action JTC
- ✓ IK funzionante sul robot reale (commit 0c5945d)
- ✓ JTC funzionante e collegato alla FSM (commit 2eb6ae6)
- ✓ Nodo `realsense_surface_node`: stima piano torso da depth ROI con PCA, pubblica frame superficie e distanza segnata TCP-piano

Impedance controller (commit 553ff7d, 2026-03-15)

- ✓ `impedance_controller_realsense.py`: mini state machine interna APPROACH (20 cm lungo normale) → HOLD (10s) → RETRACT (20 cm ritorno)
- ✓ Interfaccia `/impedance_enable` (Bool) / `/impedance_done` (Bool)
- ✓ Safe startup 3s, compensazione dinamica (gravity+coriolis), scale adattivo J2 in funzione del reach XY, Jacobiano LOCAL_WORLD_ALIGNED
- ✓ `safe_controller_switch.py`: convertito in nodo persistente con servizi ROS2 `/safe_switch/to_torque` e `/safe_switch/to_jtc`
- ✓ FSM estesa a 7 stati con ciclo JTC → torque → JTC

- ✓ Vecchi file spostati in Old/: `trajectory_manager.py`, `z1_FSM_old.py`, `z1_ik_to_jtc_old.py`, `z1_ik_to_jtc_standalone.py`

Workspace checker (commit dd6c4b7, 2026-03-15)

- ✓ `workspace_checker.py`: calcolo raggiungibilità massima con Pinocchio + scipy L-BFGS-B, clip target a (`max_reach` - 30 cm)
- ✓ Stato `CHECKING_WORKSPACE` nella FSM (thread non bloccante, `ThreadPoolExecutor`)
- ✓ Target latchato al lancio del thread: il robot completa sempre il movimento anche se il tracker perde il lock
- ✓ Topic `/target_out_of_workspace` (Bool) per integrazione futura con Spot
- ✓ Rimossi gli abort per target non fresco in `APPROACHING` e `WAIT_IK_DONE`

HOMING + EMERGENCY + keyboard safety (commit 22c15a2, 2026-03-15)

- ✓ Stato iniziale FSM: `HOMING` (porta il braccio in `home_position` all'avvio, poi `WAITING`)
- ✓ Stato `EMERGENCY_SWITCHING`: forza switch JTC se torque attivo, poi torna in `HOMING`
- ✓ Stato `EMERGENCY`: bloccante, aspetta solo `reset` da tastiera
- ✓ Nuovo nodo `z1_keyboard_safety.py`: raw terminal mode (termios), tasti H=Home, ESC=Emergency, R=Reset, Q=Quit; refresh stato 2Hz
- ✓ Parametri YAML per home pose: `home_position`, `home_orientation`
- ✓ Launch file `z1_control.launch.py`: `safe_controller_switch` + `z1_ik_to_jtc` + `impedance` (`t=0s`), FSM (`t=5s` delay)
- ✓ Launch file `z1_perception.launch.py`: YOLO tracker + surface node

4 TODO — Da completare

Attività in sospeso

Alta priorità

- ☐ **Calibrare la home position reale**: il valore attuale in `z1_fsm_params.yaml` è un placeholder (`[0.3, 0.0, 0.3]`, quaternione identità). Verificare con il robot reale qual è la posa di riposo sicura e aggiornare il YAML.
- ☐ **Testare la sequenza completa sul robot reale**: `HOMING` → `WAITING` → `CHECKING_WORKSPACE` → `APPROACHING` → `WAIT_IK_DONE` → `SWITCHING_TO_TORQUE` → `IMPEDANCE_RUNNING` → `SWITCHING_TO_JTC` → `WAITING`
- ☐ **Validare il workspace checker** con target reali: verificare che il clip a (`max_reach` - 30 cm) sia corretto e che la posa clippata sia effettivamente raggiungibile

- ☐ **Testare `z1_keyboard_safety`** sul robot reale: verificare che H, ESC e R funzionino in tutti gli stati FSM (inclusi gli stati torque)

Media priorità

- ☐ **Integrazione con Spot / base mobile:** usare `/target_out_of_workspace` per comandare alla base di avvicinarsi quando il target è fuori workspace
- ☐ **Timeout e recovery in IMPEDANCE_RUNNING:** aggiungere timeout di sicurezza per il caso in cui il ciclo impedance non termini (es. `/impedance_done` non arriva mai)
- ☐ **Tuning parametri impedance controller:** guadagni cartesiani, distanza di approach, durata hold (10s configurabile?)
- ☐ **`z1_keyboard_safety` nel launch file:** valutare se aggiungere un modo per avviarlo automaticamente in un terminale dedicato (es. `xterm` o `gnome-terminal` nel launch file)

Bassa priorità / futuri miglioramenti

- ☐ **Logging e monitoraggio:** aggiungere ROS bag recording automatico al launch con i topic principali
- ☐ **Visualizzazione RViz:** marker per il target, workspace reachability, piano superficie
- ☐ **Multi-persona:** gestione del caso in cui YOLO rileva più persone (attualmente prende la prima/più vicina)
- ☐ **Calibrazione estrinseci camera-braccio:** verificare e documentare la trasformazione TF tra frame camera e frame base Z1

5 Sequenza di avvio

Prerequisito: il workspace ROS2 deve essere compilato e sourced.

```
cd ~/Documents/GIT_Repositories/TERESA_ws
colcon build --packages-select z1_vision && source install/setup.bash
```

Terminale 1 — Hardware robot + camera + TF

```
# Avvia driver hardware Unitree Z1, RealSense D435, TF statiche
ros2 launch z1_vision z1_realsense.launch.py
```

Attende che il `JointTrajectoryController` sia attivo e la camera streaming.

Terminale 2 — Percezione (YOLO + surface)

```
ros2 launch z1_vision z1_perception.launch.py

# Opzionale: solo tracker senza surface node
# ros2 launch z1_vision z1_perception.launch.py use_surface_node:=false
```

Terminale 3 — Stack di controllo

```
# Avvia: safe_controller_switch + z1_ik_to_jtc + impedance (t=0s)
#          z1_FSM dopo 5s (primo stato: HOMING)
ros2 launch z1_vision z1_control.launch.py

# Opzionale: aumentare il delay se serve piu' tempo
# ros2 launch z1_vision z1_control.launch.py fsm_delay:=8.0
```

Attende il completamento dell'HOMING prima di passare in WAITING.

Terminale 4 — Keyboard safety (interattivo, separato)

```
ros2 run z1_vision z1_keyboard_safety
```

Tasto	Azione
H	Invia braccio alla home position (qualunque stato)
ESC	Emergenza: stop immediato, poi HOMING
R	Reset dallo stato EMERGENCY
Q	Chiude il nodo keyboard safety

Monitoraggio (terminali aggiuntivi opzionali)

```
# Stato FSM in tempo reale
ros2 topic echo /z1_fsm/state

# Target YOLO
ros2 topic echo /torso_target_ee_locked

# Distanza TCP-superficie
ros2 topic echo /surface_signed_distance

# Target fuori workspace
ros2 topic echo /target_out_of_workspace

# FK end-effector
ros2 run z1_vision fk_reader
```

6 File principali

File	Ruolo
z1_FSM.py	FSM principale (11 stati)
z1_ik_to_jtc.py	IK Pinocchio → JTC action
impedance_controller_realsense.py	Controllo impedenza cartesiana 500 Hz
safe_controller_switch.py	Switch JTC ↔ torque (servizi ROS2)
workspace_checker.py	Raggiungibilità workspace (Pinocchio + scipy)
z1_yolo_torso_tracker.py	YOLO11 + Kalman 3D + lock
realsense_surface_node.py	Stima piano torso (PCA depth ROI)

File	Ruolo
z1_keyboard_safety.py	Safety node interattivo da terminale
kalman_filter.py	Filtro Kalman 3D [x,y,z,vx,vy,vz]
fk_reader.py	Utility: FK end-effector da joint_states
config/z1_fsm_params.yaml	Parametri FSM (home pose, topic, workspace)
config/z1_ik_jtc_params.yaml	Parametri IK/JTC (URDF, tolleranze, traiettoria)
config/z1_yolo_torso_params.yaml	Parametri YOLO/Kalman/lock
config/impedance_control_params.yaml	Parametri impedance (guadagni, timing)
config/surface_params.yaml	Parametri surface node
launch/z1_control.launch.py	Avvia stack controllo (switch + IK + impedance + FSM)
launch/z1_perception.launch.py	Avvia stack percezione (YOLO + surface)
launch/z1_realsense.launch.py	Avvia HW robot + camera + TF

7 Note tecniche

- **IK:** Jacobiana pseudo-inversa smorzata (*damped least squares*), frame LOCAL, interpolazione smoothstep quintica (zero velocità a inizio/fine), joint unwrap con `_make_target_near` per evitare rotazioni $> |\pi|$
- **Kalman:** dt adattivo dal timestamp immagine, `vel_damping` = 0.9, covarianza noise configurabile da YAML
- **Lock tracker:** varianza soglia su 20 frame + 5 check consecutivi sotto soglia
- **Impedance:** 500 Hz, PID cartesiano posizione + joint-space polso (J4–J6), compensazione gravity+coriolis, Jacobiano LOCAL_WORLD_ALIGNED, scale adattivo J2 in funzione del reach XY
- **URDF path** (hardcoded):
`/home/andrea/Ros2_repositories/unitree_z1_ws/install/
z1_description/share/z1_description/urdf/z1.urdf`
- **Branch corrente:** FSM_main (fare merge su main dopo i test sul robot reale)